CS527 – Computer Systems: OpenMP Lab Report (Arrays and Matrices)

Name: Soudip Das · Roll no.: 2026CSM1037

1. Setup and method

Item	Value
Language	C++ (standard C++ with OpenMP)
Compiler	g++ 16.2.0, MSYS2 UCRT64 (Windows)
Compile command	`g++ -fopenmp -O2 -Wall taskN_xxx.cpp -o taskN` (no warnings) Eg: `g++ -fopenmp -O2 -Wall task1_vecadd.cpp -o task1`
Machine	[CPU model : Intel(R) Core(TM) Ultra 5 225H (1.70 GHz)], [RAM : 32.0 GB (31.4 GB usable)]. OpenMP reports 14 hardware threads (`omp_get_max_threads() = 14`)
Timing	`omp_get_wtime()`
Setting the number of threads	**In code, with `omp_set_num_threads(n)`**. Each program loops over 1, 2, 4 and 8 threads itself
Correctness	Every task has a serial and an OpenMP version. Outputs are compared before timings are reported (exact comparison for integers and exactly-representable doubles, tolerance 1e-9 for matrix multiplication). The `Match` / `Correct` columns below are all **YES**
Data layout	Tasks 4 and 5 use flattened 1D vectors: element (i, j) is `A[i * n + j]`

Speedup = T_serial / T_parallel. **Efficiency** = speedup / number of threads.

Measurement caveat. The timer on this Windows setup only ticks in steps of about 1 ms (many times are exact multiples of 0.001 s). Runs shorter than a few milliseconds are therefore unreliable (they show `nan`, `inf` or 0.00 speedups). These are marked *n/a* or noted in the text. Each configuration was run once, so small differences (a single 1 ms tick) should not be over-interpreted. All conclusions below rely on the larger sizes.

2. Task 1 – Vector addition

c[i] = a[i] + b[i] with a[i] = i and b[i] = 2i (exact in double). Serial loop versus `#pragma omp parallel for`. Element-wise `==` comparison: **all match**.

N = 10^5 (serial 0.000000 s): all parallel times are 0.000–0.001 s, so below timer resolution. Speedup: *n/a*.

N = 10^6 (serial 0.001 s): parallel times 0.001, 0.001, 0.000, 0.001 s for 1, 2, 4, 8 threads. Speedup: *n/a* (equal to one timer tick).

N = 10^7 (serial 0.010 s)

Threads	Parallel (s)	Speedup	Match
1	0.009	1.11	YES
2	0.006	1.67	YES
4	0.005	2.00	YES
8	0.004	2.50	YES

N = 10^8 (serial 0.094 s): this is the speedup table for the largest N

Threads	Parallel (s)	Speedup	Match
1	0.097	0.97	YES
2	0.067	1.40	YES
4	0.043	2.19	YES
8	0.039	2.41	YES

Explanation of the speedup. At N = 10^8 the speedup grows with threads but only reaches 2.41 on 8 threads, far from 8. Vector addition does one addition per 24 bytes moved (two reads, one write), so it is limited by memory bandwidth, not by the number of cores.

Q: At what array size does parallelization start paying off? Why not for small N? From the data, around N = 10^7 (speedup 1.1 to 2.5, though coarse) and clearly at N = 10^8 (up to 2.4x). For N = 10^5 and 10^6 the whole serial loop takes about a millisecond or less. Creating or waking threads and splitting the loop costs microseconds (Task 4 shows 5 to 40 µs per parallel region here), which is comparable to the work itself, so there is nothing to gain.

Q: Does speedup keep increasing linearly with threads? No. Going from 1 to 2 threads gives 1.4x, from 2 to 4 gives 1.6x more, and from 4 to 8 only 1.1x more. The memory system saturates after a few threads, so extra threads just wait for data. Thread start-up overhead and the fact that the machine's 14 hardware threads are not all full cores also limit scaling.

3. Task 2 – Sum of array elements

The array is `a[i] = (i % 10) + 1` (ints) and the sum is kept in `long long`. The correct sum for N = 10^7 is 55,000,000.

Step 1: naive parallel sum (data race), N = 10^7, 4 threads

Run	Naive (no reduction)	`reduction(+:sum)`	Naive correct?
1	13,750,000	55,000,000	NO
2	13,750,000	55,000,000	NO
3	13,750,000	55,000,000	NO
4	13,750,000	55,000,000	NO
5	13,750,000	55,000,000	NO

The naive version is wrong in every run. Here it gave the same wrong value each time, exactly one quarter of the right answer. With `-O2`, each thread accumulates its own chunk in a register and writes `sum` back at the end, so the last thread to finish overwrites

the others and only one thread's chunk (13,750,000) survives. A data race has undefined behaviour, so on another machine, compiler setting, or thread count the wrong answer can also change from run to run.

Step 2: reduction, timed. Result matches the serial sum on every run (Correct = YES).

N = 10^7 (serial 0.004 s)

Threads	Reduction (s)	Speedup	Correct
1	0.004	1.00	YES
2	0.002	2.00	YES
4	0.002	2.00	YES
8	0.001	4.00	YES

N = 10^8 (serial 0.043 s)

Threads	Reduction (s)	Speedup	Correct
1	0.046	0.93	YES
2	0.024	1.79	YES
4	0.011	3.91	YES
8	0.008	5.37	YES

Step 3: `critical` instead of `reduction` (N = 10^7, serial 0.004 s)

Threads	Critical (s)	Reduction (s)	Critical correct
1	0.129	0.005	YES
2	0.266	0.002	YES
4	0.215	0.001	YES
8	0.426	0.001	YES

Explanation of the speedup. The reduction scales well (5.4x on 8 threads at N = 10^8) because each thread sums privately and the partial sums are combined once. It scales better than vector addition because it only reads one array and writes nothing back. The `critical` version is correct but much slower than even the serial loop.

Q: Why does the naive version give wrong or inconsistent answers? `sum += a[i]` is really three steps: read `sum`, add `a[i]`, write `sum`. All threads share the single variable `sum`, so two threads can read the same old value, each add their own number, and write back, and one update is lost. This is a data race. In our run the compiler's register optimisation made it lose three quarters of the updates every time.

Q: Would `critical` fix correctness? What happens to performance? Yes, it makes the answer correct (the table shows YES), because only one thread at a time may execute `sum += a[i]`. But the loop is now effectively serial plus the cost of locking on every iteration, and threads queue up. Measured at N = 10^7, `critical` is about 25 times slower than `reduction` on 1 thread and several hundred times slower on 8 threads, and it gets slower as threads are added (0.129 s at 1 thread to 0.426 s at 8 threads) because of lock contention. Even 1 thread with `critical` is far slower than the plain serial loop (0.129 s versus 0.004 s).

4. Task 3 – Maximum and minimum with index

Random integers (fixed seed). Three ways of finding the value or the (value, index) pair: `reduction(max/min)` (value only), a `critical` section updating a shared pair, and a manual reduction (each thread scans its own chunk, results are combined in a serial pass). If a value appears more than once, all versions return the smallest index, so every result is identical to the serial version. All rows: **Correct = YES**.

All times in seconds. Red = `reduction` (value only), Crit = `critical` (value + index), Man = manual per-thread reduction (value + index).

N = 10^7. Serial: max = 999999993 at index 9044659 (0.006 s); min = 67 at index 4002523 (0.005 s).

Threads	Max Red	Max Crit	Max Man	Min Red	Min Crit	Min Man	Correct
1	0.006	0.123	0.003	0.007	0.121	0.002	YES
2	0.004	0.263	0.002	0.003	0.274	0.002	YES
4	0.003	0.303	0.001	0.002	0.312	0.001	YES
8	0.002	0.404	0.001	0.002	0.404	0.001	YES

N = 10^8. Serial: max = 999999999 at index 50737336 (0.052 s); min = 3 at index 40908195 (0.051 s).

Threads	Max Red	Max Crit	Max Man	Min Red	Min Crit	Min Man	Correct
1	0.056	1.205	0.027	0.054	1.210	0.025	YES
2	0.029	2.722	0.015	0.029	2.630	0.017	YES
4	0.016	3.125	0.010	0.015	2.639	0.011	YES
8	0.010	3.573	0.008	0.011	3.646	0.008	YES

Speedup of the maximum at N = 10^8 (serial 0.052 s) for 1, 2, 4, 8 threads: reduction 0.93, 1.79, 3.25, 5.20; manual 1.93, 3.47, 5.20, 6.50; critical 0.04, 0.02, 0.02, 0.01. The minimum behaves the same way.

Explanation of the speedup. The reduction and manual versions scale well (5 to 6.5x on 8 threads) because threads work independently and meet only once at the end. The `critical` version is slower than serial at every thread count, and gets worse with more threads.

Q: Why can't we write `reduction(max: best_idx)` to get the index? A reduction combines one variable at a time. `reduction(max: best_idx)` would return the largest index value that any thread saw, which is simply the last index in the array, not the index where the maximum value occurs. We need the value and its index to travel together as a pair, and a plain `max` reduction on one variable cannot do that.

Q: `critical` versus manual per-thread reduction: which is faster, and why? The manual version is far faster: at N = 10^8 with 8 threads it takes 0.008 s against 3.573 s for `critical`, about 450 times faster. In the `critical` version, every one of the 10^8 iterations has to enter a lock, so threads wait for each other and the loop is serialised with locking overhead on top (it is even about 69 times slower than the serial loop at 8 threads). In the manual version each thread runs a normal, lock-free loop on its own chunk with local variables, and only the handful of per-thread results are combined at the end.

5. Task 4 – Matrix addition

Serial, parallel outer loop only, and `collapse(2)`. Matrices are flattened 1D vectors. Times are averages over repeated runs, in microseconds. All results match the serial matrix (Correct = YES).

N = 8 (serial 0.05 µs; averaged over 20,000 repetitions)

Threads	Outer (µs)	Collapse (µs)	Speedup (outer)	Speedup (collapse)	Correct
1	5.80	5.50	0.01	0.01	YES
2	10.45	10.80	0.00	0.00	YES
4	24.65	25.80	0.00	0.00	YES
8	38.15	39.40	0.00	0.00	YES

N = 4096 (serial 16,599.99 µs, about 16.6 ms)

Threads	Outer (µs)	Collapse (µs)	Speedup (outer)	Speedup (collapse)	Correct
1	16,200.02	19,399.98	1.02	0.86	YES
2	11,600.02	11,999.99	1.43	1.38	YES
4	7,399.99	7,600.02	2.24	2.18	YES
8	8,400.01	9,200.00	1.98	1.80	YES

Explanation of the speedup. For N = 8 the work is 64 additions (about 50 nanoseconds), while starting a parallel region costs several microseconds and grows with the number of threads, so parallel is thousands of times slower than serial. For N = 4096 the speedup is only about 2.2 at best (4 threads) and falls at 8 threads, because matrix addition is memory-bound, like Task 1.

Q: For small N, does `collapse(2)` help? (iterations available to threads) In our measurements, no: the outer and collapse times are almost identical (for example 38.15 µs and 39.40 µs on 8 threads). In principle `collapse(2)` turns 8 independent iterations (one per row) into 64 (one per element), which matters when threads outnumber rows (this machine has 14 threads; with 8 rows only 8 of them could ever get work in the outer-only version) or when the rows cannot be split evenly. But here the total work is so tiny that the cost of the parallel region dominates, and that cost is the same for both versions, so splitting the work more finely cannot help.

Q: For large N, does `collapse(2)` make a measurable difference? No. With N = 4096 the outer loop already has 4096 iterations to share among at most 8 threads, which keeps every thread busy. The collapsed version is even slightly slower (0.86 versus 1.02 on one thread, 1.80 versus 1.98 on 8), probably because of the extra index arithmetic needed to recover i and j from the collapsed index.

6. Task 5 – Matrix–matrix multiplication

A and B are N x N doubles stored as flattened 1D vectors. Versions: serial i-j-k; parallel on the outer `i` loop; `collapse(2)` on (i, j); and the bonus loop order i-k-j (serial and parallel on `i`). Each cell below is **time (s) / speedup / efficiency**. Speedup for the i-j-k versions is relative to serial i-j-k; for the i-k-j version it is relative to serial i-k-j. All versions matched the serial result (Correct = YES for every row).

N = 256 (serial i-j-k 0.0120 s, serial i-k-j 0.0050 s)

Threads	outer-i	collapse(2)	outer-i (i-k-j)
1	0.0130 / 0.92 / 0.92	0.0130 / 0.92 / 0.92	0.0050 / 1.00 / 1.00
2	0.0080 / 1.50 / 0.75	0.0060 / 2.00 / 1.00	0.0030 / 1.67 / 0.83
4	0.0040 / 3.00 / 0.75	0.0030 / 4.00 / 1.00	0.0010 / 5.00 / 1.25
8	0.0040 / 3.00 / 0.38	0.0030 / 4.00 / 0.50	0.0010 / 5.00 / 0.62

At N = 256 the times are only 1 to 13 timer ticks, so these numbers are rough (for example, the efficiency of 1.25 is a timer artefact).

N = 512 (serial i-j-k 0.1130 s, serial i-k-j 0.0370 s)

Threads	outer-i	collapse(2)	outer-i (i-k-j)
1	0.1150 / 0.98 / 0.98	0.1100 / 1.03 / 1.03	0.0360 / 1.03 / 1.03
2	0.0580 / 1.95 / 0.97	0.0600 / 1.88 / 0.94	0.0200 / 1.85 / 0.92
4	0.0280 / 4.04 / 1.01	0.0280 / 4.04 / 1.01	0.0100 / 3.70 / 0.92
8	0.0190 / 5.95 / 0.74	0.0190 / 5.95 / 0.74	0.0060 / 6.17 / 0.77

N = 1024 (serial i-j-k 2.8450 s, serial i-k-j 0.3120 s)

Threads	outer-i	collapse(2)	outer-i (i-k-j)
1	2.5990 / 1.09 / 1.09	2.5650 / 1.11 / 1.11	0.2930 / 1.06 / 1.06
2	0.8520 / 3.34 / 1.67	0.8450 / 3.37 / 1.68	0.1560 / 2.00 / 1.00
4	0.4440 / 6.41 / 1.60	0.4420 / 6.44 / 1.61	0.0800 / 3.90 / 0.98
8	0.3090 / 9.21 / 1.15	0.3080 / 9.24 / 1.15	0.0390 / 8.00 / 1.00

Explanation of the speedup. Matrix multiplication does O(N^3) work on O(N^2) data, so it is compute-heavy and scales much better than the earlier tasks (up to 9.2x for the i-j-k version and 8.0x for the i-k-j version on 8 threads at N = 1024). For N = 1024 the i-j-k efficiency is above 1 (up to 1.68). Efficiency above 1 is not magic: the serial i-j-k baseline walks down a column of B on every inner iteration, which is hard on the cache, and it is a single run. Splitting the rows across several cores also gives the threads more cache between them. The i-k-j version, which reads memory in order, has efficiency of about 1.0 throughout, which supports this explanation. Treat the exact efficiency values above 1 with caution.

Bonus: i-k-j loop order. Swapping the loops makes the serial code much faster: 0.012 to 0.005 s (N = 256), 0.113 to 0.037 s (N = 512) and 2.845 to 0.312 s (N = 1024), about 9 times faster at N = 1024. The reason is that the inner loop now walks along rows of B and C, which are contiguous in memory, instead of jumping down a column of B. The parallel i-k-j version at 8 threads takes 0.039 s, about 73 times faster than serial i-j-k at N = 1024. At N = 1024 its efficiency stays at about 1.0, while the i-j-k efficiency is above 1 (see the explanation above). In absolute terms i-k-j wins by a wide margin at every thread count.

Q: Why is it safe to parallelize the `i` loop even though `k` sums into `C[i][j]`? Each value of `i` writes only to its own row C[i][0..N-1] and reads A and B, which nobody modifies. Different threads therefore never touch the same element of C, so there is no race. The summation over `k` happens entirely inside one thread (in its own local `sum`).

Q: What happens if the `k` loop is parallelized with a plain `parallel for`? All threads would add their products into the same `C[i][j]` (or the same local `sum`) at once. This is exactly the lost-update data race of Task 2: some additions are overwritten, so the result is wrong and can change between runs. It would need `reduction(+:sum)`. This is also why `collapse(2)` is applied only to `i` and `j` and not to `k`: every (i, j) pair writes a different `C[i][j]`, but all `k` iterations for one (i, j) write the same one.

Q: `parallel for` on `i` versus `collapse(2)` on (i, j). For which N does `collapse(2)` matter most? It matters when N is small compared with the number of threads (for example N = 4 or 8 with 8 to 14 threads), because the outer loop alone then has fewer iterations than threads, and also when N is not a multiple of the thread count, so the last round of rows is unevenly shared. With N = 512 and 1024 there are far more rows than threads, and our measurements show practically identical times for both versions (for example 0.309 s versus 0.308 s on 8 threads at N = 1024). At N = 256 the collapse version looks a little faster (0.003 s versus 0.004 s), but that is one timer tick and not conclusive.

Note on the local `sum`. Accumulating into a local variable and writing `C[i][j]` once keeps the running total in a register, so the inner loop does not read and write memory on every iteration. It also avoids having several threads write to the same memory location.

7. Overall conclusions

- Parallelism pays off only when there is enough work per thread: tiny problems (Task 1 at 10^5 and Task 4 at N = 8) are slower in parallel because of overhead.
- Memory-bound problems (vector addition, matrix addition) scale poorly (about 2 to 2.5x here) because memory bandwidth is shared, while compute-heavy matrix multiplication scales close to the number of threads.
- A shared variable updated by several threads needs a `reduction` (or per-thread local copies). `critical` is correct but orders of magnitude slower, and it gets slower as threads are added.
- Memory access order matters as much as parallelism: the i-k-j loop order made the serial matrix multiplication about 9 times faster at N = 1024.